C# Ders - 12 Ekim 2026 Ders Notları

1. Modern Karar Yapıları: Switch Statement vs. Switch Expression

C#'ta çoklu eşitlik kontrollerinde `if-else` merdivenleri yerine `switch` yapıları kullanılır.

A. Geleneksel `switch-case` (Statement)

Eski ve klasik yaklaşımdır. Her durum için `case` blokları açılır, işlem tamamlandığında akış `break;` komutuyla sonlandırılır:

C#

```
switch (day.ToLower())
{
    case "pazartesi":
        Console.WriteLine("monday");
        break;
    case "salı":
        Console.WriteLine("tuesday");
        break;
    default:
        Console.WriteLine("Bilinen evrende böyle bir gün yok!");
        break;
}
```

- `default`: Belirtilen durumların hiçbiri sağlanmadığında çalışacak yedek bloğu temsil eder.
- `break;`: Koşul sağlandığında bloğu terk etmek için zorunludur.

B. Modern Switch Expression (`=>` ve `_`)

C# 8.0 ile gelen bu yapı, kodu bir deyim (expression) haline getirerek doğrudan bir değişkene değer döndürmeyi sağlar. Daha kısa, temiz ve fonksiyoneldir:

C#

```
string month = UserSay.ToLowerInvariant() switch
{
    "ocak" or "1" or "şubat" or "2" or "aralık" or "12" => "Kış",
    "mart" or "3" or "nisan" or "4" or "mayıs" or "5"   => "İlkbahar",
    "haziran" or "6" or "temmuz" or "7" or "ağustos" or "8" => "Yaz",
    "eylül" or "9" or "ekim" or "10" or "kasım" or "11"  => "Sonbahar",
    _ => "Yok Öyle Bir Ay", // Discard / Default karşılığı
};
```

Özellik	Geleneksel `switch-case`	Modern Switch Expression
Sözdizimi	`switch (degisken) { case x: ... break; }`	`degisken switch { deger => sonuc, }`
Geriye Değer Dönme	Doğrudan dönemez; değişkene içeride atama yapılır	Doğrudan bir değişkene atanabilir (`string sonuc = ...`)
Varsayılan Durum	`default:`	`_` (*Discard / Alt çizgi*)
Veya (`or`) Kullanımı	Alt alta boş case'ler yazılır (`case 1: case 2:`)	`or` anahtar sözcüğü ile tek satırda birleştirilir

C. Çoklu Eşleşme Mantığı (`or` Deseni)

Switch Expression içinde aynı sonuca gidecek durumlar mantıksal `or` anahtar kelimesiyle tek satırda toplanabilir:

C#

```
"pazartesi" or "pzt" => "Monday",
"salı" or "sal"      => "Tuesday",
```

Kullanıcı hem günün uzun adını hem de kısaltmasını girdiğinde aynı sonuç üretilir.

2. Metin Karşılaştırmalarında Kültür Problemi: `ToLowerInvariant()`

Kod bloğunda dikkat çeken `UserSay.ToLowerInvariant()` metodu, diller arası harf dönüşüm hatalarını önler:

- `ToLower()`: İşletim sisteminin bölgesel ayarlarına (Culture) göre çalışır. Türkçe bir sistemde büyük `'I'` harfi küçük `'ı'` harfine dönüşür; `'i'` harfi **olmaz**. Bu

durum `"INPUT".ToLower() == "input"` kontrolünün Türkçe sistemlerde patlamasına yol açar.

- `ToLowerInvariant()`: İşletim sisteminin dilinden bağımsız, evrensel İngilizce/kültürsüz (Invariant) kurallara göre küçültme yapar. Kodun her sunucuda ve her kullanıcıda tutarlı çalışması için sistem/kod içi kontrollerde tercih edilir.

3. Zaman Yönetimi: `DateTime` Yapısı

`DateTime`, tarih ve saat bilgilerini tutan bir değer türüdür (`struct`).

C#

```
DateTime now = DateTime.Now; // Sistemin o anki tarih ve saatini alır
```

Önemli `DateTime` Özellikleri (Properties)

Özellik	Döndürdüğü Değer	Açıklama
`now.Year`	`int` (Örn: `2026`)	Yıl bilgisi
`now.Month`	`int` (1 - 12)	Yılın kaçıncı ayı olduğu
`now.Date`	`DateTime` (`13.10.2026 00:00:00`)	Saati sıfırlanmış sadece tarih kısmı
`now.Hour`	`int` (0 - 23)	Saat bilgisi
`now.Minute`	`int` (0 - 59)	Dakika bilgisi
`now.Second`	`int` (0 - 59)	Saniye bilgisi
`now.DayOfWeek`	`DayOfWeek` (Enum)	Haftanın günü (`Tuesday`, `Monday` vb.)
`now.DayOfYear`	`int` (1 - 366)	Yılın kaçıncı gününde olunduğu

Özel Tarih Nesnesi Tanımlama

`new` anahtar kelimesiyle belirli bir güne ait nesne üretilebilir:

C#

```
DateTime ay = new DateTime(now.Year, now.Month, now.Day);
// Yıl, ay, gün verilerek saat kısmı 00:00:00 olan yeni bir tarih üretilir.
```

4. Zaman Farkı ve Aralıklar: `TimeSpan` Yapısı

İki `DateTime` nesnesi birbirinden çıkarıldığında sonuç bir tarih **değil**, iki an arasındaki **süre farkıdır**. Bu süre farkı `TimeSpan` türünde saklanır.

Tarih1−Tarih2=TimeSpanTarih1−Tarih2=TimeSpan

C#

```
DateTime input = new DateTime(2000, 1, 1);
TimeSpan fark = DateTime.Now - input;
```

Kod İncelemesi ve Hassas Yaş Hesabı

Kodundaki yaş hesaplama yaklaşımı:

C#

```
Console.Write("Doğum Tarihini Gir: ");
DateTime.TryParse(Console.ReadLine(), out DateTime input);

TimeSpan ts = DateTime.Now - input;

// Gün bazında tam yaş hesabı:
int yas = (int)(ts.TotalDays / 365.25);
Console.WriteLine($"{yas} Yaşındasın");
```

> **Önemli İpucu:** Kodunda `ts / 365` ifadesi doğrudan derlenmez veya `TimeSpan` nesnesinin gün sayısını değil toplam süresini 365'e bölmeye çalışır. Gün bazlı doğru hesaplama için `ts.TotalDays` kullanılmalıdır (artık yılları hesaba katmak için `365.25` böleni tercih edilir).

Yıl Farkı ile Yaş Hesabı:

Sadece doğum yılını bilmek yetiyorsa:

C#

```
int dogumYili = 2004;
int yas = DateTime.Now.Year - dogumYili;
```

Bu yaklaşım pratik olsa da kişinin o yıl içindeki doğum günü henüz gelmediyse yaşı 1 yaş büyük gösterebilir. Gün/ay hassasiyetinde `DateTime` farkı almak en doğru sonucu verir.

5. Bütünsel Özet Tablosu

```
[ Kullanıcı Girişi ]
        |
        ▼
.ToLowerInvariant()  ──> Kültür bağımsız küçük harfe çevirme
        |
        ▼
switch expression    ──> "deger" or "kisaltma" => Sonuc
        |
        ▼
DateTime.Now         ──> Sistem anlık zamanı
        |
        ▼
Tarih Farkı (-)      ──> TimeSpan (TotalDays, TotalHours)
```